CSS:

É possível utilizar o CSS de três formas: interno, externo e em linha.
Em linha, adicionamos junto do objeto HTML.
Exemplo: <h1 style="font: 'arial';"> *título* </h1>

De modo interno, adicionamos um <style></style> dentro do arquivo HTML.
De modo externo, criamos um arquivo .css para as propriedades dos objetos HTML.

Para usar as propriedades, usamos os id's e classes. Caso usemos apenas os nomes, sem id's e classes, as propriedades serão usadas em todos objetos com o mesmo nome.
Exemplo: se eu tenho duas seções, ambas herdarão as propriedades.

Seletores para indicar objetos:

- **#nomedoobjeto { propriedades };** *(# - para ID)*
- **.nomedoobjeto { propriedades };** *(. - para classes)*

Seletores para acessar objetos dentro de heranças:

- **div p {};** *(adiciona propriedades a todos os parágrafos de div (netos e filhos))*
- **div > p {};** *(adiciona propriedades apenas aos parágrafos diretos de div (filhos))*
- **div + p {};** *(primeiro elemento que vem logo depois de um outro específico)*
- **div ~ p {}; (***Estiliza todos os elementos do mesmo nível que vêm depois de um outro específico.)*

Seletor universal:

- *** {};**

Para comentar no código:
/* comentário */

Cores:

Podemos colocar cores com: RGB, HSL, HEX(#) e nomes.
Exemplos:

```
#ffcccc
rgb(255, 204, 204)
hsl(0, 100%, 90%)
```

No fundo, podemos aplicar gradiente de cores:
Exemplo:

```
background: linear-gradient(to right, #00cc00 0%, #6600ff 100%);
```

Aqui temos um gradiente linear para a direita, da cor verde para o roxo.

Fontes:

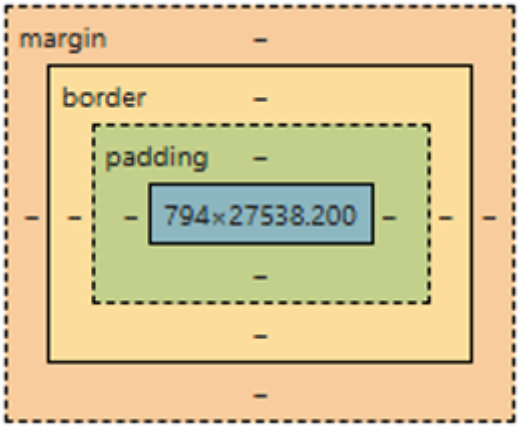
– font-family: Define o tipo de fonte.
– font-size: Define o tamanho da fonte. Pode ser em (px), (em) ou (%).
– font-weight: Define a espessura da fonte. Normalmente valor numérico de 100 a 900 ou nomes (bold, normal).
– text-align: Alinha o texto em relação ao seu contêiner.

Fundos:
Para acessar o fundo, usamos o background.
– background-color: Define a cor de fundo.
– background-image: Define uma imagem de fundo.
– background-size: Ajusta o tamanho da imagem de fundo.

Box Model:

O Box Model é o formato de um objeto HTML. Todo objeto HTML é um contêiner retangular.

Esse contêiner tem 4 partes:
Conteúdo: Conteúdo real do objeto.
Padding: Espaço entre o conteúdo e a borda. Serve para afastar o conteúdo da borda. Fica dentro da caixa junto do conteúdo.
Border: É a borda ao redor do padding e do conteúdo.
Margin: É o espaço fora da caixa, separando um elemento de outro. Serve para fazer espaçamento de objetos.

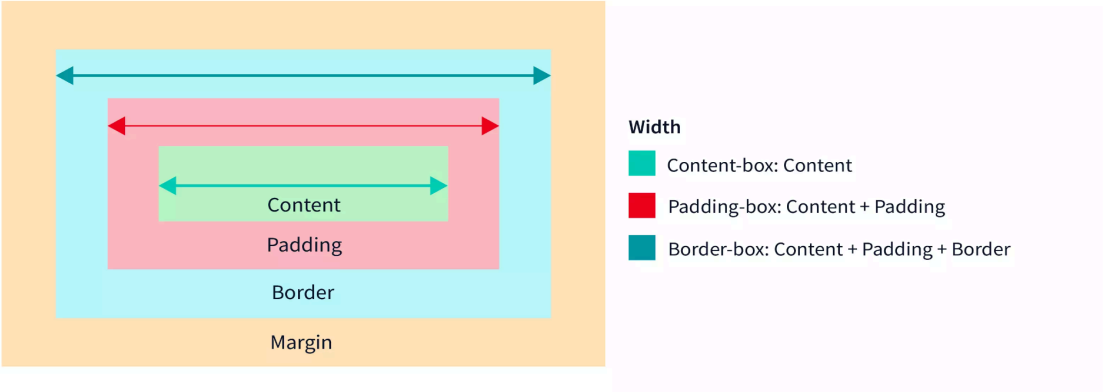


É importante dizer que existe uma propriedade que calcula automaticamente o tamanho do contêiner, o *box-sizing.*

Podemos usar *box-sizing* com: *content-box, border-box* e *padding-box.*

O content-box é o padrão.

Quando você define *height* e *width*, o *box-sizing: content-box* insere esses valores somente ao conteúdo. Já quando você usa *box-sizing: padding-box,* ele define os valores de: conteúdo + padding, e quando você usa *box-sizing: border-box*, ele define os valores de: conteúdo + padding + border.



! importante: o border-box é melhor para os layouts.

Display:

Define a forma como o elemento será exibido na página. Permite os seguintes comportamentos:

Inline: Elemento posicionado em linha.
block: Elemento ocupa toda toda a largura disponível.
flex: Elemento tem posição flexível.
grid: Elemento posiciona em grade/matriz.

inline:
– Mantém-se na mesma linha de outros elementos inline.
– Não permite definir altura e largura.

block:
– Sempre inicia em uma nova linha.
– Ocupa toda a largura disponível.
– Permite definir altura, largura, margens e paddings.

inline-block:
– Mantém-se na mesma linha.
– Permite definir altura e largura.

flex:
– Facilita o alinhamento dinâmico dos elementos.
– Trabalha com espaçamento automático entre itens.

grid:
– Divide a página em colunas e linhas. Ideal para layouts completos.

tag : tag usada para definir partes específicas sem mudar elemento todo.

Position:

Usado para definir a posição do elemento.
5 valores principais:
– static (padrão):
O elemento segue o fluxo normal da página, sem alterações de posição.
O elemento fica na posição normal, sem deslocamento por *top, left, right* e *bottom.*

– relative:
O elemento pode ser deslocado em relação à sua posição original.
O elemento ainda ocupa o espaço original no layout, mas visualmente ele se desloca.

– absolute:
O elemento é removido do fluxo normal e posicionado em relação ao primeiro elemento pai que tenha *position*: *relative, absolute ou fixed.*

– fixed:
O elemento é fixado na tela e não se move ao rolar a página.
Muito usado para menus fixos, botões de "voltar ao topo", etc.

– sticky:
O elemento se comporta como relative até atingir um determinado ponto da tela, onde então ele se fixa.